

9th Jun, 26

Databricks

(1)

In AWS, you build a data platform by wiring together many separate services. Each is its own console, its own permissions model, its own billing.

In Databricks: we collapse all of that into one platform - the "Lakehouse".

One workspace, one governance layer, one storage format, one compute engine, used for ETL, warehousing, streaming, ML & AI Agents.

* Pillars of Databricks

1. Delta Lake -

- table format
- ACID transactions, time travel on top of files in cloud
- Every table you make is a
- AWS parallel - Apache Iceberg

* Volumes - UC governed folders for non-tabular files (CSVs, images, models) (governed S3 prefix)

* Medallion architecture is followed

* SQL warehouse is compute tuned for BI/SQL.

* Cluster is general compute for notebooks/jobs.

2. Unity Catalog - (UC)

- One place for all perms, discovery across tables, ML models, dashboards.
- Namespacing - 'catalog.schema.table'
top container, think like database server
↳ database
↳ table only
- AWS parallel - Lake Formation + Glue Data Catalog + IAM

3. Compute -

- Clusters (managed spark VMs) / Serverless
- Photon is their fast C++ query engine.
- AWS parallel - EMR / Glue & Athena Serverless

4. Workspace + Notebooks -

- Web UI where you write Notebooks, build pipelines, dashboards & jobs.
- AWS parallel - Glue Studio + SageMaker Studio + Athena + Quicksight

⊛ ETL & Ingestion

1. Source files land in a volume (Unity catalog's home for non-tabular files)

→ Lakeflow connect (live connection) - you pick a source authenticate once, & Databricks runs a fully managed serverless pipeline that lands data as UC tables. - no cluster to size, no script. Managed first-party connectors are Salesforce, Service Now, GA4, SQL Server.

• For databases WITHOUT a managed connector, you still have plain JDBC reads. [Java Database connectivity]

Closest to AppFlow & DMS on AWS

→ AutoLoader (batch read)

Files already in (or arriving in) obj storage are pulled from a location.

• No crawler step, like in AWS - Auto Loader infers schema on ingest & evolves it automatically, when new columns appear.

→ Lakehouse Federation

• data stays in the source (Postgres, Snowflake^{etc}) - Data Bricks does not copy or store it.

• Federated table → data lives in another system; UC holds a pointer to it - Queried live, no store.

→ streaming sources

read directly from Kafka, Kinesis, Event Hubs with structured streaming

→ Plain Batch reads using file uploads/COPY INTO

→ Delta sharing - receive data someone else publishes to you across orgs/platforms, no copying.

≡ Imperative → a notebook with spark code

↓
Declarative → you list every step in order, and need to run them in right sequence.

[Lakeflow Declarative Pipeline - DLT]

Declarative → we declare how each table looks & its quality rules, Databricks figures out the ~~rules~~ rules

DAG

① Data Warehousing (Lakehouse → SQL warehouse → AI/BI)

→ In Databricks, there is no separate warehouse to load data into

→ Difference from classic AWS pattern - with Redshift you maintain two systems: curated data in S3, then a copy loaded into Redshift's own storage for fast BI.

→ Databricks collapses that: one copy of the data in open Delta format on your object storage, queried by both Spark for ETL & SQL for BI, governed by one UC.

Athena (serverless SQL over S3) - closest to Databricks SQL

→ AI/BI dashboards

→ elastic like Athena (not live cluster)

- Photon (vectorized C++ query engine) - on by default for SQL warehouse) liquid clustering & Z-ordering on Delta tables (the rough analog of dist/sort keys in Redshift) colocates data using common filter columns.
- Cost: Databricks serverless SQL Warehouse bills DBU for the time it's actually up - it spins down on IDLE.

* Streaming

- Recently a new native option called Zerobus Ingest ~~data~~ is available. (Feb 23, 2026)
- Serverless, Kafka-free ingestion service that lets applications & IoT devices stream data straight into Delta Lake with low latency.
- [only an option when you don't need a broker in b/w - i.e. not suitable when multiple consumers need same events or we need to rewind & replay a stream]

* CI/CD

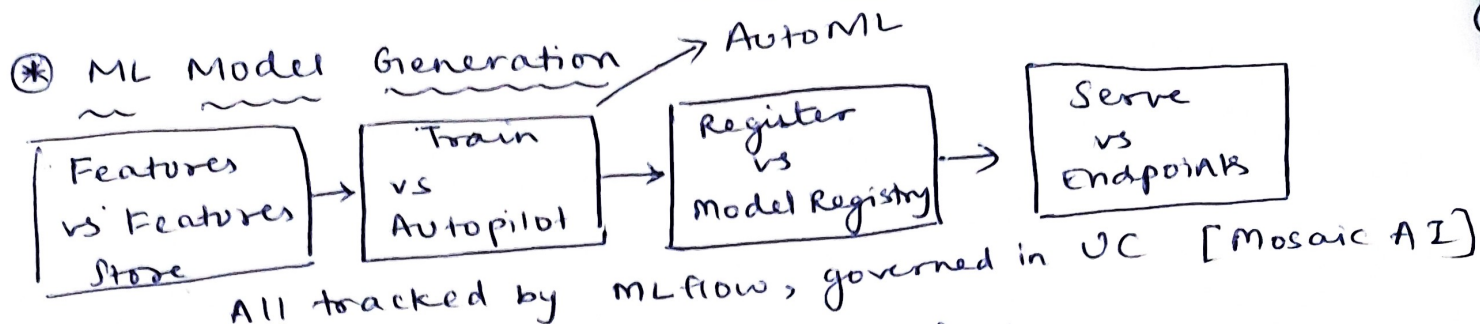
- Unit of deployment is a bundle (Databricks Asset Bundle / DAB) - recently rebranded "Declarative Automation Bundle")

- A bundle is a folder with that declares your jobs, pipelines, ML models as code, alongside the Terraform for Databricks assets under the hood, b
- write Terraform.

IaC - Infra as code - manage infra by writing config in file rather than console.

Terraform - most widely used IaC tool made by HashiCorp. Cloud agnostic. uses its own language (HCL) & tracks what it has created in a statefile so it knows what to add, change or delete on the next run.

- Key Data Bricks specific Idea → write the bundle once but you can define multiple targets that point at diff. workspace or catalogs with diff. settings



→ Data & ML live in the same place.

→ MLflow tracks every experiment (params, metrics, artifacts) & holds the model registry, which now lives in unity catalog.

→ Two ways to train:

① AutoML

- fastest

- point it at a table & target a column, trains many models & hands you the best one PLUS the generated notebook → i.e. you get editable code unlike AutoPilot (AWS)

② manual training code (pick algo, features, etc) & let MLflow ~~record~~ record what happened.

* AI agents (vs Bedrock)

→ Genie -

- no code

- scope a genie space to some tables & people ask ques in plain Eng - it writes & runs the SQL.

→ Agent Bricks -

- describe the task in plain language & connect your data; it auto-generates evaluations & optimizes the agent for quality & cost.

→ Mosaic AI Agent Framework

- full control
 - build a RAG or tool calling agent in code, evaluate & deploy
 - Vector search - turns your documents/data into embeddings for retrieval (BedRock knowledge Base)
 - MLFlow 3.0 - agent tracing, evaluation, monitoring - even for agents running outside databricks.
 - Model Serving - hosts the agent as an endpoint
- # NOTE - AI functions in SQL like ai_query or ai_classify can call an LLM straight from a SQL query.
-

* Catalog (top level containers)

↓
schemas are logical containers under catalog

↓
tables, volumes, models & stored functions